Aktiiviset tietokannat

Palvelimella sijaitsevaa toiminnallisuutta

Oppimistavoitteet

1. Opitaan liipaisimien, tapahtumien ja tallennettujen proseduurien toimintaperiaate.
2. Ymmärretään, milloin em. aktiivista tietokantatoiminnallisuutta on tarkoituksenmukaista käyttää.

Lähtökohta

- Eheyden ylläpito on yleensä tärkeimpiä syitä relaatiotietokannan käyttöönotolle.
- Datan eheys edellyttää erilaisten rajoitteiden voimassa pysymistä.
- Kolme keskeistä rajoitetyyppiä.
  - Avainrajoite
  - Arvoaluerajoite
    - Tietoyypit, CHECK-rajoitteet, NOT NULL jne.
  - Viite-eheysrajoitteet
- ANSI SQL-92-standardiin määritettiin lisäksi assertiot, joiden avulla voidaan ilmaista monimutkaisia rajoitteita.

Assertiot

- Assertiot määrittävät tietokannan reunaehtoja, joita yhden tai useamman tietokannan taulun tulee jatkuvasti täyttää.
- Assertiot ovat mukana SQL-92- ja SQL-99-standardeissa.
- Kautta aikojen yksikään tiedonhallintaohjelmisto ei ole tukenut niitä.
- Oracle on kuitenkin toteuttanut assertiot versiossa 23.26.1, joka julkaistiin tammikuussa 2026.
  - https://docs.oracle.com/en/database/oracle/oracle-database/26/sqlrf/create-assertion.html
  - Oraclen vaatimukset CHECK-osan jälkeiselle ehdolle ovat hieman tiukemmat kuin standardissa esitetään, sillä käytössä on ainoastaan eksistentiaali- ja universaaliehto (EXISTS, NOT EXISTS). Silti suuri joukko käyttötapauksia voidaan toteuttaa niiden avulla.
- MariaDB ei tue assertioita.
- Vastaavaa toiminnallisuutta voidaan usein toteuttaa muiden tekniikoiden avulla.
  - Näihin tutustutaan seuraavaksi.

Palvelimella sijaitsevat kyselyt

- Tähän asti on tarkasteltu tilanteita, jossa tiedonhallintaohjelmisto saa suoritettavakseen mielivaltaisen SQL-lauseen.
- Operaatiot voidaan myös tallentaa palvelimella.
- Tällaisia palvelimella tallennettavia operaatioita ovat
  - Tallennetut proseduurit
  - Liipaisimet
  - Tapahtumat
- Liipaisemat ja tapahtumat ovat aktiivisten tietokantojen työvälineitä.
  - Aktiivisella tietokannalla tarkoitetaan tietokantaa, joka reagoi tapahtumiin kuten datan päivityksiin.

Tallennetut proseduurit (*stored procedures*)

- Tallennettu proseduuri sijaitsee valmiiksi palvelimella.
- Kyselyn käynnistävä sovellus
  - Kutsuu tallennettua proseduuria nimen perusteella
  - Välittää parametrit
  - Käsittelee mahdollisen tulosjoukon

Esimerkki: Tallennetut proseduurit

```
delimiter //
CREATE PROCEDURE tilaushaku (IN p1 INT)
BEGIN
SELECT * FROM TILAUS WHERE Tilausnumero = p1;
END //
delimiter ;
```

- Tallennettu proseduuri suoritetaan CALL-lauseella:

```
CALL tilaushaku(123);
```

Edellä kyselyn loppumerkki joudutaan tilapäisesti vaihtamaan, jottei SELECT-lauseen lopussa oleva puolipiste katkaise proseduurin määritystä.

Tallennettujen proseduurien etuja

- Tehokkuus
  - Kerran jäsennettyä ja optimoitua kyselyä voidaan käyttää uudelleen.
- Läpinäkyvyys sovellusten suuntaan.
  - Muuttaminen ei vaikuta dataa käyttäviin sovelluksiin.
- Lisää tietoturvaa.
  - Tallennetut proseduurit ovat parametrisoituja, valmiiksi kirjoitettuja kyselyjä, joihin on (oikein käytettynä) vaikea kohdistaa esimerkiksi SQL-injektioita.
  - Voidaan antaa EXECUTE-oikeus proseduurien suorittamiseen, mutta jättää antamatta UPDATE/INSERT/DELETE-oikeudet.

Haittapuolena on jäykkyys: tallennettu proseduuri on aina vakiomuotoinen.

SQL-injektio

- Yllä on sovelluksen (esim. PHP-ohjelma web-palvelimella) merkkijonoista rakentama SQL-kysely.
- Normaalisti jälkimmäinen merkkijono olisi esimerkiksi web-lomakkeelta saatu tilausnumero (vaikka 1048).
- Vihamielinen käyttäjä voisikin syöttää sovellukselle merkkijonon **`3 OR (1=1)`**

SQL-injektio

```
DELETE FROM TILAUS WHERE Tilausno = 3 OR (1=1) ;
```

- Yllä olevan kyselyn WHERE-osassa on kaksi vaihtoehtoa (OR), joista oikeanpuoleinen on identtisesti tosi.
- Tämä vastaa kyselyä: DELETE FROM TILAUS;
- Tällaisilta hyökkäyksiltä suojaavat
  - Lomakekenttien validointi
  - Kyselyiden parametrisointi
    - Esimerkiksi vain INT-parametrin antaminen valmiiseen kyselyyn.
- Tallennetut proseduurit toteuttavat kyselyiden parametrisoinnin palvelimella.

Liipaisimet (*triggers*)

- Liipaisimet aikaansaavat kyselyn automaattisen suorituksen, kun siihen liitetyssä taulussa oleva data muuttuu.
- Lisäys-, poisto- tai päivitysoperaatio toimii liipaisimen laukaisijana.
- Liipaisin käynnistää lisäys-, poisto- tai päivitysoperaation.

Esimerkki liipaisimesta (MariaDB)

```
CREATE TRIGGER uusitilaus
AFTER INSERT ON TILAUS
FOR EACH ROW
INSERT INTO TILAUSLOKI VALUES
(NEW.Tilausnumero, NEW.Loppusumma, NOW());
```

Liipaisimen luonti (MariaDB)

```
CREATE TRIGGER liipaisimen_nimi

BEFORE          INSERT          ON taulun_nimi
AFTER           UPDATE
                DELETE

FOR EACH ROW

lause;
```

Liipaisinten kääntöpuoli

- Liipaisinten avulla voidaan toteuttaa monia sovelluslogiikkaan kuuluvia seikkoja.
  - Rikkoo kerrosarkkitehtuuria.
  - Tekee järjestelmästä vaikeammin muokattavan ja virhealttiimman.
- Eheysrajoitteet ovat ensisijainen eheyden ylläpitokeino liipaisimiin verrattuna.
  - Esim. ON DELETE CASCADE -määre viite-eheysmäärityksessä määrää johdannaispoiston.
- Liipaisimien käyttö on perusteltua silloin, kun se edistää datan eheyttä.
  - Esimerkiksi varmistetaan välttämättömien rajoitteiden voimassapysyminen.

Tapahtumat (*events*)

- Tapahtumat sallivat SQL-operaatioiden ajastamisen tapahtuvaksi
  - tiettynä aikana
  - määräajoin
- Tapahtumien avulla voidaan automatisoida määräaikainen ylläpitotoimia kuten
  - Varmuuskopiointi
  - Tilastotiedon kirjoittaminen lokiin

Esimerkki tapahtumasta (MariaDB)

```
CREATE EVENT minuuttiloggaus
ON SCHEDULE EVERY 1 MINUTE
DO
INSERT INTO MINUUTTILOKI VALUES (NOW());
```

Tapahtuman luonti (MariaDB)

```
CREATE EVENT tapahtuman_nimi
ON SCHEDULE aikataulu
DO
lause;
```

- MariaDB:ssä erillinen tapahtumien skedulointisäie vastaa tapahtumien käynnistämisestä
  - Käynnistettävä järjestelmämuuttujan **`event_scheduler`** kautta, jos pois päältä

Case: keskeytyneiden tilausten poisto

tilausnumero	asiakasnumero	loppusumma	maksu_alkoi	maksu_valmis
1	47	35,45	2023-11-14 12:33:52	(NULL)
2	93	106,65	2023-11-14 12:33:58	(NULL)
3	88	924,3	2023-11-14 12:34:03	(NULL)

- **`Tilaus`**-taulussa on kentät maksutapahtuman alkamiselle ja päättymiselle.
- Asiakas ohjataan maksuntarjoajan palveluun maksutapahtuman ajaksi.
  - Samalla kirjataan **`maksu_alkoi`**-aikaleima.

Case (jatkoa)

tilausnumero	asiakasnumero	loppusumma	maksu_alkoi	maksu_valmis
1	47	35,45	2023-11-14 12:33:52	2023-11-14 12:35:30
2	93	106,65	2023-11-14 12:33:58	(NULL)
3	88	924,3	2023-11-14 12:34:03	2023-11-14 12:35:24

- Kun maksutapahtuma päättyy onnistuneesti, kirjataan aikaleima **`maksu_valmis`**.
- Miten tunnistetaan keskeytyneet maksutapahtumat?
  - Asiakas on voinut esim. sulkea selaimen.

Case (jatkoa)

```
DELIMITER //
CREATE EVENT poista_vanhat_tilaukset
ON SCHEDULE EVERY 1 MINUTE
DO
BEGIN
    DELETE FROM TILAUS
    WHERE maksu_alkoi IS NOT NULL
      AND maksu_valmis IS NULL
      AND maksu_alkoi < NOW() - INTERVAL 10 MINUTE;
END;
//
DELIMITER ;
```

- Ajastettu tapahtuma skannaa tilaukset läpi määräajoin.
  - Tilaukset, joissa maksunvälittäjän takaama maksimiaika on ylittynyt, poistetaan.

Case (jatkoa)

tilausnumero	asiakasnumero	loppusumma	maksu_alkoi	maksu_valmis
1	47	35,45	2023-11-14 12:33:52	2023-11-14 12:35:30
3	88	924,3	2023-11-14 12:34:03	2023-11-14 12:35:24

- Keskeytynyt tapahtuma on poistunut, kun ajastettu tapahtuma on suoritettu.